

Q27. Case Study: Cryptocurrency Price Tracker (Quick Sort)

A cryptocurrency platform tracks and sorts prices dynamically for thousands of coins. Analyze how Quick Sort can be applied for fast sorting. Identify issues such as pivot selection and fluctuating data. Apply divide and conquer concepts to explain its working. Analyse performance and propose optimizations with proper justification.

Describe the Real-World Problem

Cryptocurrency exchanges such as Binance, Coinbase, and Kraken manage thousands of cryptocurrencies whose prices change every second due to market demand and supply. Traders need to quickly identify the highest-priced, lowest-priced, or top-performing cryptocurrencies for making investment decisions.

Since cryptocurrency prices fluctuate continuously, the platform must frequently sort large amounts of price data. If the sorting process is slow, users may receive outdated information, affecting trading decisions. Therefore, an efficient sorting algorithm is required to organize cryptocurrency prices quickly and accurately.

Quick Sort is an efficient Divide and Conquer algorithm that can rapidly sort large datasets, making it suitable for cryptocurrency price tracking systems.

Problem Statement

Design a cryptocurrency price tracking system that sorts cryptocurrency prices efficiently using the **Quick Sort algorithm**.

The system should:

- Accept cryptocurrency names and their prices.
- Sort the prices in ascending or descending order.
- Handle thousands of cryptocurrency records efficiently.
- Minimize the time required for sorting.
- Display the sorted list accurately even when prices are updated frequently.

Algorithm Used

Quick Sort Algorithm (Divide and Conquer)

Quick Sort is a Divide and Conquer sorting algorithm that recursively divides the dataset into smaller parts.

Steps

1. Select a pivot element.
2. Partition the array into:
 - Elements smaller than the pivot.
 - The pivot element.
 - Elements greater than the pivot.

3. Apply Quick Sort recursively to both partitions.
4. Continue until every partition contains one or zero elements.
5. Combine all partitions to obtain the sorted array.

Why This Algorithm?

Quick Sort is chosen for the Cryptocurrency Price Tracker because the application must sort thousands of cryptocurrency prices quickly as market values change continuously. It provides an average time complexity of **$O(n \log n)$** , making it highly efficient for large datasets. Quick Sort also performs **in-place sorting**, requiring only **$O(\log n)$** extra stack space, which reduces memory usage and improves execution speed.

Comparison: Quick Sort vs Merge Sort

Feature	Quick Sort	Merge Sort
Average Time Complexity	$O(n \log n)$	$O(n \log n)$
Worst Case Time Complexity	$O(n^2)$	$O(n \log n)$
Extra Space Required	$O(\log n)$	$O(n)$
In-Place Sorting	Yes	No
Practical Performance	Faster in most real-world applications	Slightly slower due to merging and extra memory
Suitable for Cryptocurrency Price Tracker	Yes	Yes, but uses more memory

Why Quick Sort Instead of Merge Sort?

Although Merge Sort guarantees $O(n \log n)$ performance even in the worst case, it requires additional memory to create temporary arrays during the merging process. In a cryptocurrency price tracker, where prices are updated frequently and thousands of records must be sorted repeatedly, minimizing memory usage is important.

Quick Sort performs sorting within the original array (in-place), uses less memory, and is generally faster in practice because of better cache performance. Therefore, Quick Sort is the preferred algorithm for this application, as it provides an excellent balance of speed, efficiency, and low memory consumption.

Complexity Analysis

Case	Time Complexity
Best Case	$O(n \log n)$
Average Case	$O(n \log n)$
Worst Case	$O(n^2)$
Space Complexity	$O(\log n)$

Explanation

- **Best Case:** Pivot divides the array equally.
- **Average Case:** Nearly balanced partitions.
- **Worst Case:** Pivot is always the smallest or largest element.
- Recursive calls require **$O(\log n)$** stack space.

Manual Process (Step-by-Step Execution)

Cryptocurrency Prices

Cryptocurrency	Price (\$)
Bitcoin	108000
Ethereum	3800
Solana	190
XRP	2.50
Dogecoin	0.25
BNB	720
Cardano	1.20

Initial Array

[108000, 3800, 190, 2.50, 0.25, 720, 1.20]

Step 1

Choose **1.20** as Pivot.

Left:

[0.25]

Pivot:

1.20

Right:

[108000,3800,190,2.50,720]

Step 2

Sort Right Partition

Choose **720** as Pivot.

Left:

[190,2.50]

Pivot:

720

Right:

[108000,3800]

Step 3

Sort Left Partition

Choose **2.50** as Pivot.

Left:

[]

Pivot:

2.50

Right:

[190]

Sorted

[2.50,190]

Step 4

Sort Right Partition

Choose **3800** as Pivot.

Left:

[]

Pivot:

3800

Right:

[108000]

Sorted

[3800,108000]

Final Sorted Array

[0.25,1.20,2.50,190,720,3800,108000]

Applications

Quick Sort is used in:

- Cryptocurrency exchanges
- Stock market applications
- Banking systems
- Financial analytics
- Database management systems
- Search engines
- E-commerce product sorting
- Data analytics
- Scientific computing
- Operating systems

Advantages

- Very fast for large datasets.
- Average complexity is **$O(n \log n)$** .
- Uses Divide and Conquer strategy.
- In-place sorting algorithm.
- Requires less memory.
- Better practical performance than many sorting algorithms.
- Suitable for real-time applications.
- Easy to implement recursively.

Limitations

- Worst-case complexity is **$O(n^2)$** .
- Performance depends on pivot selection.
- Not a stable sorting algorithm.
- Recursive calls increase stack usage.
- Frequent price updates may require repeated sorting.
- Poor pivot selection reduces efficiency.

Comparison with Other Algorithms

Algorithm	Best	Average	Worst	Space	Suitable for Large Data
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Small Data
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	Yes

Justification: Although Merge Sort guarantees $O(n \log n)$ in the worst case, it requires additional memory. Quick Sort is generally faster in practice because it sorts in place and has lower memory overhead, making it suitable for real-time cryptocurrency price tracking.

INPUT

```
def partition(arr, low, high):
    pivot = arr[high][1]
    i = low - 1

    for j in range(low, high):
        if arr[j][1] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
    arr[i + 1], arr[high] = arr[high], arr[i + 1]
    return i + 1

def quick_sort(arr, low, high):
    if low < high:
        pi = partition(arr, low, high)
        quick_sort(arr, low, pi - 1)
        quick_sort(arr, pi + 1, high)

n = int(input("Enter the number of cryptocurrencies: "))
crypto = []
for i in range(n):
    name = input(f"\nEnter Cryptocurrency {i+1} Name: ")
    price = float(input(f"Enter {name} Price ($): "))
    crypto.append([name, price])

print("\nOriginal Cryptocurrency Prices")
print("-" * 40)
for item in crypto:
    print(f"{item[0]:15} : ${item[1]:.2f}")
quick_sort(crypto, 0, len(crypto) - 1)
print("\nSorted Cryptocurrency Prices (Ascending)")
print("-" * 40)
for item in crypto:
    print(f"{item[0]:15} : ${item[1]:.2f}")
```

Output:

```
Enter the number of cryptocurrencies: 7

Enter Cryptocurrency 1 Name: bitcoin
Enter bitcoin Price ($): 14200

Enter Cryptocurrency 2 Name: Ethereum
Enter Ethereum Price ($): 3800

Enter Cryptocurrency 3 Name: Solana
Enter Solana Price ($): 190

Enter Cryptocurrency 4 Name: XRP
Enter XRP Price ($): 2.5

Enter Cryptocurrency 5 Name: dogecoin
Enter dogecoin Price ($): 45

Enter Cryptocurrency 6 Name: xrp
Enter xrp Price ($): 203

Enter Cryptocurrency 7 Name: bnb
Enter bnb Price ($): 2500

Original Cryptocurrency Prices
-----
bitcoin      : $14,200.00
Ethereum     : $3,800.00
Solana       : $190.00
XRP          : $2.50
dogecoin     : $45.00
xrp          : $203.00
bnb          : $2,500.00
```

```
Sorted Cryptocurrency Prices (Ascending)
-----
XRP          : $2.50
dogecoin     : $45.00
Solana       : $190.00
xrp          : $203.00
bnb          : $2,500.00
Ethereum     : $3,800.00
bitcoin      : $14,200.00
```

Conclusion

Quick Sort is an efficient Divide and Conquer algorithm for sorting large collections of cryptocurrency prices. Its average time complexity of $O(n \log n)$ enables fast and efficient sorting, making it suitable for real-time cryptocurrency trading platforms. Although the algorithm has a worst-case complexity of $O(n^2)$ due to poor pivot selection, this issue can be minimized by using optimized pivot selection techniques such as Random Pivot or Median-of-Three Pivot. Overall, Quick Sort provides an excellent balance of speed, memory efficiency, and practical performance, making it an ideal choice for cryptocurrency price tracking applications.